

DOCUMENTO TECNICO RISERVATO • SVILUPPO

Brief di prodotto e piano di lavoro.

Stato attuale, tre casi pilota reali, architettura target e roadmap per la prosecuzione dello sviluppo della piattaforma OVIA a partire dalla codebase esistente.

DESTINATARIO

Max — sviluppatore incaricato

COMMITTENTE

L3 Innovation S.r.l.
P.IVA 02882330901

DATA

15 luglio 2026

VERSIONE

1.0 — base di discussione

Come leggere questo documento

Questo non è un capitolato chiuso. È la fotografia onesta di dove siamo, di cosa mi hanno chiesto tre clienti reali, e di cosa penso vada costruito. Le sezioni 09 e 10 contengono le domande a cui mi serve una tua risposta prima di aprire l'editor.

01 Cos'è OVIA

Il prodotto in una frase. Cosa vendiamo e cosa no.

3

02 Il punto di partenza: la codebase esistente

Cosa c'è già, cosa manca, e la decisione che chiedo di non riaprire.

4

03 I tre pilota

Mainardi, Tommaso, Studio [C] — chi sono, cosa vogliono, cosa pagano.

7

04 Il denominatore comune

Perché non costruiamo tre software. Core + moduli.

10

05 Specifica funzionale dei moduli

M1-M6. Cosa fa ognuno, dove sta già nel codice, cosa manca.

12

06 Architettura tecnica target

Stack, pipeline AI, orchestrazione, dati.

17

07 Guard rail — cosa non facciamo

I limiti che tengono in piedi la vendita.

18

08 Roadmap operativa

Sprint 0-4 e task ereditati dal precedente sviluppatore.

19

09 Debito tecnico da sanare

Le otto criticità che bloccano l'uso su dati reali.

21

10 Domande aperte e prossimo passo

Cosa mi serve da te, cosa ti serve da me.

23

Cos'è OVIA

IL PRODOTTO IN UNA FRASE

OVIA è il **cervello operativo** di uno studio professionale: raccoglie tutto ciò che entra — chiamate, mail, documenti, scadenze — lo indicizza, lo pesa e lo restituisce quando serve, senza dimenticare niente.

Il brand si è chiamato Praxis fino a luglio 2026. Ora si chiama **OVIA**. Il sito è in costruzione, il posizionamento è: *«L'AI costruita sul tuo studio. Il tuo tempo, restituito.»* Ci rivolgiamo a studi professionali (commercialisti, avvocati, consulenti), liberi professionisti e PMI.

La regola di posizionamento che vincola anche il prodotto

Non usiamo mai la parola «gestionale». Non è una questione di marketing, ha conseguenze tecniche dirette: il cliente ha già TeamSystem o Zucchetti e non li sostituisce. OVIA non compete con il gestionale, ci si *affianca* e lo *legge*. Ogni scelta di prodotto che ci porta a rifare l'anagrafica fiscale, la contabilità o il libro giornale è una scelta sbagliata: quel terreno è perso in partenza.

OVIA È

- ✓ Il livello intelligente sopra i dati dello studio
- ✓ Memoria: nulla si perde, tutto è interrogabile
- ✓ Priorità: dà il giusto peso alle cose
- ✓ Braccio destro: propone e prepara, l'umano conferma
- ✓ Rivendibile a N studi con configurazione, non con riscrittura

OVIA NON È

- ✗ Un gestionale contabile o fiscale
- ✗ Un sostituto di TeamSystem / Zucchetti / Profis
- ✗ Un chatbot generico brandizzato
- ✗ Un progetto custom diverso per ogni cliente
- ✗ Un sistema che manda mail o prende decisioni senza conferma umana

VINCOLO COMMERCIALE CHE SI TRADUCE IN VINCOLO TECNICO

Il modello di business è **una piattaforma, N studi**. Ogni funzionalità che chiedo va progettata multi-tenant e configurabile da UI. Se una richiesta di un cliente non è generalizzabile, è mio compito rifiutarla — ma se ti accorgi che sto per farti scrivere codice specifico per un singolo studio, **fermami**.

Il punto di partenza: la codebase esistente

Ricevi in allegato il `pacchetto-consegna-nexuscore.zip` : la consegna tecnica del precedente sviluppatore, datata 06/07/2026. Il codename interno del software è **NexusCore** — il brand commerciale è OVIA. Il pacchetto contiene handover tecnico, analisi funzionale, template di configurazione, dump PostgreSQL demo, procedura di deploy e documentazione della pipeline AI/RAG.

I segreti operativi (password DB, OpenAI API key, credenziali SSH e demo) te li passo su canale separato una volta definito l'accordo.

PRIMA COSA DA FARE, PRIMA ANCORA DI LEGGERE

Il pacchetto dichiara di non contenere segreti. Non è vero.

Il `README.md` afferma che il pacchetto non contiene segreti reali. Ma `08-architettura-ai/PIPELINE_RAG.md`, riga 170, contiene la chiave del webhook Plaud di sviluppo in chiaro. In più, il documento tecnico (§18) segnala che `Data/DevelopmentSeedData.cs` contiene una password demo costante.

Conseguenza operativa: quella chiave e quella password vanno **ruotate**, e il pacchetto va trattato come materiale sensibile — non inoltrarlo, non caricarlo su repo pubbliche o servizi terzi. Te lo segnalo io perché è un errore del pacchetto, non tuo.

Stato dichiarato: alpha avanzata / pre-beta

.NET 8

BLAZOR SERVER + IDENTITY +
EF CORE

PG 16

POSTGRESQL + STORAGE
PRIVATO SU FS

3

PIPELINE AI GIÀ IN
PRODUZIONE

Stack rilevato: Blazor Web App / Blazor Server, ASP.NET Core Identity, Entity Framework Core, PostgreSQL 16, MudBlazor, Nginx reverse proxy, systemd, storage documenti su filesystem privato.

```
Internet
→ Nginx HTTPS reverse proxy
→ Kestrel / ASP.NET Core su 127.0.0.1:5000
→ PostgreSQL 16 (5432)
→ document storage privato /var/www/nexuscore/documents
```

Cosa c'è, e quanto è reale

Colonna «reale» ricavata incrociando `CONTINUAZIONE_SVILUPPO.md` con `analisi-funzionale-software.md`. Dove i due documenti divergono, ho tenuto per buono il secondo, che è l'analisi diretta della codebase. Vedi il riquadro sotto.

AREA	REALE	STATO
Auth e utenti	SÌ	Identity con ruoli Admin / Staff / Client. Multi-tenant (uno studio = un tenant, <code>TenantId</code> sulle entità principali). Multi-sede con grant cross-sede e redirect a <code>/select-branch</code> .
Permessi granulari	PARZIALE	Pattern <i>permission-as-role</i> : 6 permessi definiti, auto-seed e UI di assegnazione presenti. Ma l'enforcement server-side non è completo per tutti . Policy globali limitate ad admin/staff/client/tenant, nessun handler resource-based per entità. La segregazione staff per assegnazione cliente/pratica è nel modello ma non applicata uniformemente su tutte le query .
Clienti e pratiche	SÌ	CRUD completo. Stato, categoria, priorità, scadenza. Pipeline configurabile, avanzamento stage, dettaglio con documenti, chatbot e trascrizioni.
Documenti	SÌ	Upload drag & drop, storage privato, distinzione interni/visibili al cliente, indicizzazione RAG (PdfPig → chunk → embedding). Solo PDF testuali , nessun OCR.
Plaud	PARZIALE	Webhook <code>POST /api/webhooks/plaud/{tenantId}</code> con <code>X-Api-Key</code> , analisi GPT-4o → JSON strutturato, upload manuale su <code>/calls/new</code> . Il pacchetto è però esplicito: «non va dichiarato come sistema Plaud integrato nativamente end-to-end» .
RAG	VERTICAL SLICE	Due livelli, per pratica e globale studio, con citazione fonte e pagina. Funziona, ma non è enterprise-ready — vedi M2 e il punto sulla scala.
Portale cliente	MOCK	Crea account reali, password temporanea, documenti condivisi, upload dal portale. Ma: le richieste documento sono mock non persistenti, il reinvio credenziali è simulato e non manda email vere.
Comunicazioni	NO	Esiste il permesso <code>ClientCommunicator</code> e un pannello Email/SMS/WhatsApp. Il pacchetto vieta di dichiarare email/SMS/WhatsApp reali integrati . Il pannello è un guscio.
Export dati	NO	Il ruolo <code>DataExporter</code> è definito, ma nessun servizio, endpoint o pagina di export operativo è stato rilevato . Il permesso protegge una funzione che non esiste.
Scadenze	BASE	Campi scadenza su pratiche, task e attività; dashboard mostra arretrati. Nessun calendario/scadenzario completo, nessun promemoria automatico, nessuna notifica programmata, nessuna escalation, nessuna sync calendario.
Workflow	PARZIALE	Modelli EF completi (WorkflowTemplate, WorkflowStep, WorkflowActivity, CaseActivity) + 5 template di seed. Service layer incompleto (task #80–81).
API	MINIMAL	Tre endpoint: <code>POST /api/webhooks/plaud/{tenantId}</code> , <code>POST /api/documents/{id}/ingest</code> , <code>GET /api/documents/{id}</code> . Nessuna REST pubblica, nessun OpenAPI/Swagger. Le chat AI sono invocate dai componenti Blazor lato server, non da endpoint.
Admin e audit	SÌ	Dashboard KPI per tenant, audit log ricco con metadata, <code>/admin/activity</code> per collaboratore, <code>/audit-log</code> .

DUE DOCUMENTI DEL PACCHETTO SI CONTRADDICONO

`CONTINUAZIONE_SVILUPPO.md` descrive l'indicizzazione documenti come «*chunking, embedding pgvector*» — cioè cosa fatta. `PIPELINE_RAG.md` e `analisi-funzionale-software.md` dicono l'opposto: **gli embedding sono `double precision[]`, `pgvector` non è usato, la similarity è calcolata in memoria .NET.**

Ho tenuto per vera la seconda versione. **Su questo punto poggia tutta la priorità dello Sprint 1**, quindi è la prima cosa che ti chiedo di verificare a mano nel codice. Più in generale: `CONTINUAZIONE_SVILUPPO.md` in alcuni punti descrive l'*intenzione*, non il codice. Quando due documenti divergono, vince l'analisi funzionale — e vince comunque il codice su entrambi.

DECISIONE PRESA — NON LA RIAPRO

Si continua su NexusCore. Non si riscrive.

La base copre a mio giudizio circa **metà** del prodotto che sto vendendo ai tre pilota della sezione 03 — e la metà giusta: quella noiosa e costosa (multi-tenant, sedi, anagrafiche, pratiche, documenti, audit). *Questa è una mia stima, non un dato del pacchetto: correggila dopo lo Sprint 0.*

Riscrivere in un altro stack significa perdere sei mesi e ripagare cose che ho già pagato. Se ritieni che ci siano ragioni tecniche *gravi e documentabili* per cambiare, voglio sentirle prima di firmare — non dopo. Ma la preferenza personale di stack non è una ragione.

I tre pilota

Non sono ipotesi di mercato. Sono tre persone con cui ho parlato, di cui conosco il dolore specifico, e su cui il prodotto va validato. Sono deliberatamente diversi per dimensione: 50, ~700 e 12 organizzazioni.

PILOTA A

Studio Mainardi — commercialisti

21 PERSONE

~700 CLIENTI

2 SEDI

TITOLARE NON RISPONDE

Il contesto

I numeri qui sopra vengono da miei appunti di conversazione, non da un documento dello studio. Li riconfermo prima che tu li usi per dimensionare qualcosa.

Studio strutturato, relazione entrante calda (rapporto pluriennale di mio padre con il titolare). È il pilota con il potenziale economico più alto e il ciclo di vendita più lungo. Ho fatto un incontro in presenza con demo, ho mandato una proposta, non ho avuto conferma.

Il dolore dichiarato

Non è l'acquisizione clienti — è il **sovraccarico**. Troppo che entra, niente che si perda. Il turnover del personale distrugge lo storico: quando un senior se ne va, il know-how esce dalla porta. Un junior nuovo impiega mesi a diventare operativo perché non ha accesso alla storia dei clienti.

Cosa gli serve tecnicamente

- **RAG globale studio** su tutto lo storico documentale, cross-cliente e cross-pratica — già presente nel codice, va portato a scala reale (700 clienti = decine di migliaia di chunk)
- **Cattura chiamate via Plaud** con smistamento automatico sul cliente giusto
- **Multi-sede** — già implementato
- **Onboarding**: un nuovo collaboratore deve poter chiedere «cosa so di questo cliente» e ottenere una risposta con le fonti

Il blocco commerciale, e come lo sblocco

Non serve un'altra proposta. Serve fargli vedere il sistema che gira **sui suoi dati reali**: trial breve con Plaud+webhook attivo su un sottoinsieme di pratiche. Questo ha una conseguenza tecnica precisa: **mi serve un percorso di onboarding tenant che si esegua in ore, non in settimane**. È un requisito di prodotto, non un favore.

PILOTA B

Tommaso — segreteria di 12 ordini professionali

12 ORDINI

~3 ORE/GIORNO SU MAIL

HA GIÀ PLAUD

INTERESSATO, TRATTA SUL PREZZO

Il contesto

Imprenditore che gestisce in outsourcing la segreteria di 12 ordini professionali (sanitari e tecnici). La sua persona chiave operativa è **Alessandra**, che fa il lavoro quotidiano — è lei il vero utente del sistema, e va coinvolta nei test. Ciascun ordine è un'organizzazione con regole proprie.

Il dolore dichiarato

La protocollazione — registrazione formale di ogni mail e PEC in entrata/uscita sul portale nazionale — c'era ma il volume è risultato basso. Il dolore vero, emerso in riunione, è un altro: **circa 3 ore al giorno complessive passate a leggere e rispondere alle mail**. Al costo pieno di un operatore, sono ~1.100–1.200 €/mese di tempo-persona bruciato a ricopiare risposte, ~13–14k l'anno. E cresce ad ogni nuovo ordine acquisito.

Cosa gli serve tecnicamente

- **Motore mail**: lettura casella, classificazione, generazione bozza di risposta calibrata sull'ordine specifico, **conferma umana obbligatoria prima dell'invio**
- **Regole per organizzazione**: 12 ordini = 12 set di regole e tono. Questo, tecnicamente, è multi-tenant applicato dentro un singolo cliente — verificare che il modello tenant regga
- Estrazione metadati per pre-compilazione protocollo (a bassa priorità, il volume non giustifica lo sforzo oggi)

INCOGNITA BLOCCANTE, MAI RISOLTA

Il portale nazionale di protocollazione **espone un'API o un import bulk?** Nessuno l'ha verificato. Non impegniamoci su nessuna architettura di protocollazione finché non lo sappiamo. Se la risposta è no, la protocollazione resta assistita, non automatica — e va detto al cliente prima, non dopo.

PILOTA C

Studio [C] — commercialista, contatto nuovo

2 DIPENDENTI

~50 CLIENTI

INGRESSO CALDO VIA DIPENDENTE DI FIDUCIA

Il contesto

«Studio [C]» è un segnaposto, non il nome reale. Lo sostituisco prima di qualsiasi uso operativo. Non è ancora un cliente e non sa che esiste questo documento.

Studio piccolo. Non ho ancora parlato con la titolare: ho parlato con la sua **dipendente di fiducia**, che è l'ingresso e conosce i processi reali. È il pilota più veloce da chiudere e da implementare — decisione singola, poca complessità, ottimo terreno di validazione prima di andare da Mainardi.

Il dolore dichiarato — è il più chiaro dei tre

- **Mail e messaggi automatici ricorrenti:** solleciti e avvisi trimestrali/semestrali, richieste documenti ai clienti. Movente esplicito: *eliminare il «non mi avevi avvisato»* — quindi serve tracciamento dimostrabile dell'avviso inviato, non solo l'invio
- **Workflow per i dipendenti** sull'attività quotidiana, in particolare pratiche INPS
- **To-do list e calendario per la titolare**, alimentati automaticamente da ciò che entra
- **Plaud integrato:** le chiamate della titolare vanno smistate e indicizzate, consultabili da telefono e da desktop
- **RAG documentale interno** agganciato al loro gestionale per recepire fatture e documenti

NOTA DI VENDITA CHE CONDIZIONA LA TECNICA

Plaud **non** va presentato come «adesso compri Plaud». Va presentato come «automatizzo anche le chiamate, con uno strumento che uso io». Conseguenza tecnica: il device è un dettaglio implementativo nostro, quindi **l'ingestion delle trascrizioni deve essere agnostica rispetto alla sorgente** — Plaud oggi, un altro domani, o un semplice upload audio. Non incastrare il modello dati su Plaud.

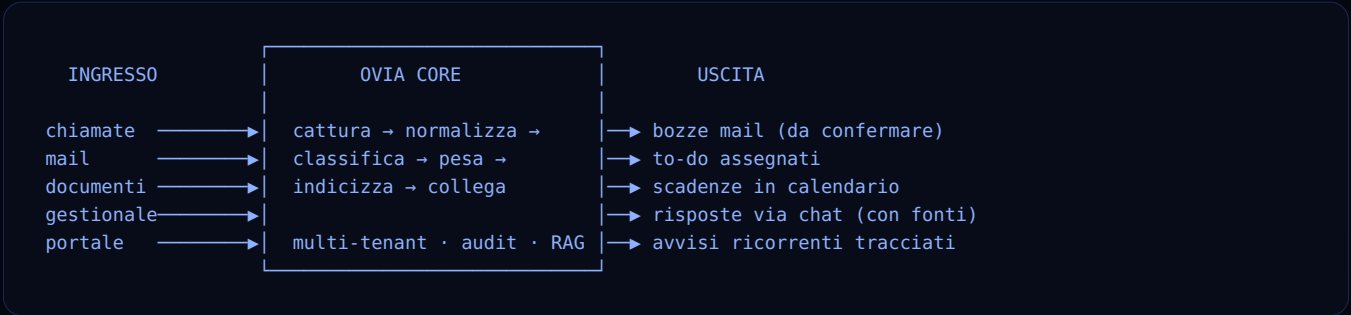
Il denominatore comune

Tre clienti, tre richieste diverse. La tentazione è costruire tre software. Sarebbe la fine del progetto: tre codebase, tre manutenzioni, zero margine.

Cosa chiedono davvero, sotto le parole diverse

BISOGNO DI FONDO	MAINARDI	TOMMASO	STUDIO [C]	TRADUZIONE
Non perdere niente	ALTO	ALTO	ALTO	Ingestion universale + indicizzazione
Interrogare lo storico	ALTO	MEDIO	ALTO	RAG documentale multilivello
Chiamate strutturate	ALTO	MEDIO	ALTO	Pipeline trascrizione → entità → pratica
Mail in uscita assistite	MEDIO	CRITICO	CRITICO	Motore mail con human-in-the-loop
Ricorrenze e scadenze	MEDIO	BASSO	CRITICO	Scheduler + template + prova d'invio
Workflow del team	ALTO	MEDIO	ALTO	Template workflow + work queue
Priorità / peso	ALTO	MEDIO	ALTO	Scoring urgenza + dashboard titolare

Le colonne convergono. Il core è uno solo: *tutto ciò che entra viene catturato, capito, pesato e reso interrogabile; tutto ciò che esce viene preparato dal sistema e confermato da un umano.* Le differenze tra i tre pilota sono differenze di **configurazione e di modulo attivo**, non di software.



La mappa moduli

ID	MODULO	STATO CODICE	CHI LO ATTIVA PER PRIMO
M0	Core: tenant, sedi, utenti, permessi, audit	FATTO	Tutti
M1	Cattura chiamate (Plaud e non solo)	FATTO, DA IRROBUSTIRE	Mainardi, Studio [C]
M2	RAG documentale (pratica + studio)	FATTO, NON SCALA	Mainardi, Studio [C]
M3	Motore mail assistito	DA COSTRUIRE	Tommaso, Studio [C]
M4	Workflow, attività e scadenze ricorrenti	MODELLI SÌ, SERVICE NO	Studio [C], Mainardi
M5	Portale cliente	PARZIALE, MAIL NON PARTONO	Mainardi (fase 2)
M6	Delega assistita e dashboard di priorità	DA COSTRUIRE	Mainardi, Studio [C]
M7	Connettori gestionali esterni	DA VALUTARE	Studio [C] (richiesto)

Specifica funzionale dei moduli

M1

Cattura chiamate

Cosa fa. Il professionista registra una chiamata o una riunione con un device (oggi Plaud). Il device produce la trascrizione. La trascrizione entra in OVIA *senza che nessuno faccia niente*, viene analizzata, agganciata al cliente e alla pratica giusti, e genera le azioni conseguenti.

Cosa c'è già. Webhook per tenant autenticato con API key. `PlaudService` chiama GPT-4o ed estrae JSON strutturato: NomeCliente, Telefono, Email, OggettoChiamata, TipoPratica, Urgenza (Bassa/Media/Alta), DataScadenza, NoteImportanti, AzioniRichieste[], Riassunto. Embedding della trascrizione salvato su `CallNote.Embedding`. Pagina `/calls/new` per l'upload manuale.

Cosa manca — in ordine di importanza

- **Matching automatico sul cliente esistente.** Oggi l'AI estrae un nome. Serve un resolver che cerchi il cliente nel tenant (fuzzy su nome + telefono + email), colleghi la `CallNote` alla pratica giusta, e se è ambiguo metta la chiamata in una coda «da attribuire» invece di indovinare. *Su 700 clienti con omonimie, indovinare è inaccettabile.*
- **Da AzioniRichieste[] a task reali.** Oggi l'array resta dentro un JSON. Deve generare `CaseActivity` assegnate, con scadenza, visibili nella work queue.
- **Astrazione della sorgente.** `PlaudWebhookController` va generalizzato in un endpoint di ingestion trascrizioni con adapter per sorgente (plaud / upload / zapier / orchestrator generico). Il modello dati non deve nominare Plaud.
- **Idempotenza e retry.** Il webhook riceve due volte lo stesso evento? Oggi crea due `CallNote`. Serve chiave di deduplica.
- **CallNote nel RAG globale.** Oggi le chiamate entrano solo nel RAG di pratica, non in quello di studio. È esattamente ciò che Mainardi comprenderebbe. **LIMITE GIÀ NOTO E DOCUMENTATO**
- **Vista mobile.** Studio [C] ha chiesto esplicitamente consultazione da telefono. Blazor Server responsive può bastare in fase 1 — da verificare, non da assumere.

```
POST /api/ingest/transcript/{tenantId} [X-API-Key]
|
| dedup(externalId) ————— già visto? → 200 no-op
| GPT-4o → entità strutturate
| ClientResolver(nome, tel, email)
|   | match unico → collega a CrmCase
|   | ambiguo/assente → coda "da attribuire"
| embedding → CallNote (indicizzata: pratica + studio)
| AzioniRichieste[] → CaseActivity (assegnate, con scadenza)
```

RAG documentale

Cosa fa. Ogni documento caricato diventa interrogabile in linguaggio naturale, con citazione della fonte e della pagina. Due livelli: dentro la singola pratica, e su tutto lo studio.

Cosa c'è già. Ingestion: PdfPig → estrazione testo per pagina → chunking 400 parole con overlap 50 → embedding batch `text-embedding-3-small` (1536 dim, max 96 input/richiesta) → `DocumentChunk`. Stato tracciato su `Document.IngestionStatus` e `DocumentIngestionJob` (Pending → Processing → Ingested / Failed / Skipped). Query: TopK 6, MinSimilarity 0.30, prompt contestuale, `gpt-4o-mini`, risposta con fonti e score.

IL PROBLEMA NUMERO UNO DI TUTTO IL PROGETTO

Gli embedding sono salvati come `double precision[]` e la **cosine similarity è calcolata in memoria .NET**, non in SQL. Funziona in demo. Non funziona con Mainardi.

700 clienti × pratiche × documenti = ordine delle decine di migliaia di chunk per tenant. Il documento di consegna dichiara degrado sopra i 5.000 chunk. **Significa che il pilota economicamente più importante è quello su cui il sistema oggi si pianta.** Questo va risolto per primo, prima di qualsiasi funzionalità nuova.

IL PROBLEMA NUMERO DUE, E NON È TECNICO — È DEONTOLOGICO

Dall'analisi funzionale: «**non risultano test di autorizzazione RAG**», non emerge un `RagAuthorizationService` completo, e «**la segregazione staff per assegnazione cliente/pratica è predisposta nel modello, ma non appare applicata in modo uniforme su tutte le query operative**».

Tradotto: la chat globale studio pesca su tutti i chunk del tenant. Se il filtro per assegnazione non è applicato ovunque, **un collaboratore può interrogare documenti di clienti che non gli competono — e ottenere risposte con le fonti citate.** In uno studio commercialista questo non è un bug, è una violazione del segreto professionale. È anche l'unico scenario in cui un difetto del nostro software diventa un problema legale del cliente. Per questo l'ho messo in Sprint 1 come bloccante e non nella lista generica dei test: **finché non è chiuso, il RAG globale non si accende su dati veri.**

L'upgrade pgvector è già stato progettato dal precedente sviluppatore, va solo eseguito:

```
CREATE EXTENSION IF NOT EXISTS vector;

ALTER TABLE "DocumentChunks"
  ALTER COLUMN "Embedding" TYPE vector(1536)
  USING "Embedding"::vector;

CREATE INDEX ON "DocumentChunks"
  USING hnsW ("Embedding" vector_cosine_ops);

-- retrieval spostato nel DB:
SELECT * FROM "DocumentChunks"
WHERE "TenantId" = @tenantId AND "CrmCaseId" = @caseId
ORDER BY "Embedding" <=> @queryVector
LIMIT 6;
```

Resto della lista M2

- Estendere embedding a note interne e attività, non solo PDF
- OCR per i PDF scansionati. L'estrazione oggi funziona **solo su PDF testuali**: nessun OCR, nessun Textract. Un PDF scansionato finisce in **Skipped** e **il cliente non lo sa** — per lui il sistema «si è dimenticato» un documento. Va almeno esposto in UI. *Mia assunzione da verificare sul campo: che la quota di scansioni in ingresso in uno studio sia alta abbastanza da renderlo prioritario. Non ho il dato.*
- Includere CallNote nel RAG globale (vedi M1)
- Nessun sistema di feedback sulla qualità delle risposte: serve almeno un pollice su/giù loggato, altrimenti non sappiamo se il RAG è buono
- Il budget OpenAI mensile (**MonthlyBudgetUsd** , default 10 \$) **oggi logga e basta, non blocca**. Con N tenant è un rischio di costo reale

M3 · DA COSTRUIRE DA ZERO

Motore mail assistito

Perché conta. È il modulo che chiude due pilota su tre, ed è l'unico con un'ancora di valore già quantificata: le 3 ore/giorno di Tommaso. Il sistema non azzerà quelle ore, le porta a supervisione: realisticamente da 3 ore a ~45 minuti di controllo bozze, quindi **~2 ore di persona liberate al giorno**. È questo il numero che vendo. Il software deve poterlo dimostrare, quindi serve strumentazione.

Requisiti

- **Connessione casella:** IMAP/Graph/Gmail API. Da decidere insieme — vedi sezione 10
- **Classificazione** del messaggio in entrata: tipo, urgenza, organizzazione/cliente di riferimento
- **Generazione bozza** con RAG sul contesto del cliente e sulle regole della singola organizzazione
- **Conferma umana obbligatoria.** Non negoziabile. Il flusso è: bozza → revisione → invio. Mai invio diretto su risposta a mail in entrata
- **Ricorrenze** (per Studio [C]): scheduler di campagne trimestrali/semestrali, template con variabili, invio su lista filtrata

- **Prova d'invio.** Requisito esplicito: il movente del cliente è chiudere il «non mi avevi avvisato». Serve log immutabile e consultabile: cosa, a chi, quando, con quale esito. Questo pezzo è audit, non marketing
- **Multi-organizzazione dentro un tenant:** i 12 ordini di Tommaso hanno regole e tono diversi. Verificare che il modello tenant/branch regga o se serve un livello «Organization»

NON FARTI INGANNARE DA QUELLO CHE VEDI A SCHERMO

Esiste il permesso `ClientCommunicator` e un pannello comunicazioni Email/SMS/WhatsApp. **Non è una base: è un guscio.** Il pacchetto vieta esplicitamente di dichiarare email/SMS/WhatsApp reali integrati. Stessa cosa per le scadenze: nessun promemoria automatico, nessuna notifica programmata, nessuna escalation, nessuna sincronizzazione calendario. **M3 e le ricorrenze si costruiscono da zero** — stimale come tali, non come completamento.

M4

Workflow, attività e scadenze

I modelli EF ci sono e sono completi. Il service layer no. Il precedente sviluppatore ha lasciato i task già scomposti (#80–#86, #91–#92): sono elencati nella sezione 08 e vanno ripresi da lì, non riprogettati.

Aggiunte necessarie per i pilota: **ricorrenze** (le scadenze fiscali sono cicliche, il modello attuale non le prevede), **work queue cross-pratica** per lo staff (task #91, richiesta da Studio [C] per i dipendenti sulle pratiche INPS), **alimentazione automatica** delle attività da M1 e M3.

M5

Portale cliente

ATTENZIONE Il documento di consegna è esplicito: credenziali simulate, **le mail non partono realmente**. Il portale è dichiarato «non prodotto finito». Non è in fase 1. Ha senso per Mainardi in fase 2, come argomento di posizionamento competitivo verso i suoi 700 clienti — non come risparmio operativo.

M6 · DA COSTRUIRE

Delega assistita e peso delle cose

È la parte che nella vendita chiamo «braccio destro» ed è la più delicata. Tradotta in requisito onesto: **il sistema propone, l'umano decide**. Non «l'AI prende decisioni importanti» — quella frase, davanti a un commercialista, la cancelliamo dal vocabolario.

- Scoring di urgenza/impatto su tutto ciò che entra (esiste già il campo Urgenza dalle chiamate, va esteso e reso configurabile)

- Dashboard titolare: cosa richiede la mia attenzione oggi, e perché — con il perché tracciabile
- Proposta di delega: «questa cosa la può gestire X» in base a carico e competenza. Suggerimento, sempre con conferma

M7 · DA VALUTARE

Connettori gestionali

Studio [C] ha chiesto di agganciare il RAG «al loro gestionale» per recepire fatture e documenti. **Non ho ancora verificato quale gestionale usino né se esponga qualcosa.** Prima di stimare, si scopre. Se non c'è API, il fallback è export periodico + cartella monitorata: brutto, ma funziona ed è vendibile. Non promettere integrazione prima della verifica.

Architettura tecnica target

Configurazione AI attuale

COMPONENTE	MODELLO	NOTA
RAG pratica + studio	<code>gpt-4o-mini</code>	Configurabile da appsettings, non hardcoded
Embedding	<code>text-embedding-3-small</code>	1536 dim, costo basso
Analisi chiamate	<code>gpt-4o</code>	Hardcoded in PlaudService — da esternalizzare
Budget	<code>MonthlyBudgetUsd: 10.00</code>	Solo log, non blocca. Consumo tracciato via <code>IAiUsageLogger</code>

CRITICITÀ ARCHITETTURALE GIÀ RILEVATA

Diversi servizi **istanzano direttamente il client OpenAI**. Serve un'astrazione provider con configurazione centralizzata. Non è purismo: senza di essa non possiamo cambiare modello, non possiamo mettere un modello EU per un cliente che lo pretende, e non possiamo fare fallback. Con commercialisti e avvocati la domanda «dove finiscono i miei dati» arriva sempre.

Pattern da conoscere prima di toccare il codice

- **Permission-as-role**. I permessi speciali sono ruoli Identity aggiuntivi, non colonne. Per aggiungerne uno: costante in `Authorization/AppRoles.cs` inclusa in `AppRoles.All`, più una riga in `PermDefs[]` di `Admin/UserEdit.razor`. Seeding, UI, toggle e salvataggio sono già gestiti.
- **DbContextFactory, non DbContext scoped**. I service che iniettano sia `UserManager` sia EF devono usare `IDbContextFactory<ApplicationDbContext>`, mai il DbContext scoped condiviso con Identity — altrimenti `InvalidOperationException: A second operation was started on this context instance`. `BranchService` è il riferimento già refactorato.
- **Blazor Server, prerender disabilitato**. Tutte le pagine interattive usano `@rendermode @(new InteractiveServerRenderMode(prerender: false))`. Non riabilitarlo senza verificare i service iniettati.
- **Titoli AppBar** mappati dall'URL in `MainLayout.razor`, metodo `PageTitle`. Ogni pagina nuova richiede il pattern corrispondente.

Dove metto la logica di orchestrazione

Commercialmente parlo di «Zapier o un orchestrator». Tecnicamente: **Zapier è il ponte per il trial, non l'architettura**. Va bene per far arrivare in fretta la trascrizione da Plaud al webhook durante un pilota — e serve proprio a quello nel trial Mainardi. Ma la logica di prodotto (matching cliente, generazione task, scoring) sta **dentro OVIA**. Se finisce in uno Zap, non è rivendibile, non è versionabile, non è testabile, e il primo cliente che disdice Zapier ci fa cadere il servizio.

Guard rail — cosa non facciamo

Il documento di consegna del precedente sviluppatore contiene un elenco di cose da non promettere. Lo faccio mio e lo estendo, perché è la lista che mi impedisce di venderti un lavoro impossibile.

NON PROMETTERE SENZA SVILUPPO DEDICATO

- ✗ Firma digitale
- ✗ Conservazione sostitutiva
- ✗ OCR completo
- ✗ Workflow legali/contabili avanzati
- ✗ Compliance GDPR completa
- ✗ SLA di disponibilità

NON CONSIDERARE FINITO

- ✗ Lo storage file come gestione documentale enterprise
- ✗ Le funzioni AI/RAG come affidabili per uso legale
- ✗ Il portale cliente come prodotto
- ✗ I permessi speciali come verificati (nessun test)

Le tre righe rosse mie

- ✗ **Nessuna mail parte senza conferma umana.** Vale per le risposte. Le campagne ricorrenti su template approvato sono l'unica eccezione, e vanno comunque approvate una volta.
- ✗ **L'AI non decide.** Propone, ordina, prepara. Il verbo «decidere» non compare in nessuna UI né in nessun contratto.
- ✗ **Nessun dato reale di cliente finale in ambienti demo o di test.** Il seed gira anche in Staging e Production (`Program.cs`) — comodo per le demo, da rivalutare prima di andare su dati veri.

Roadmap operativa

L'ordine non è casuale: prima si mette in sicurezza ciò che esiste, poi si sblocca il pilota più veloce, poi si costruisce il pezzo che chiude due clienti su tre. Mainardi arriva quando il sistema regge il suo volume — non prima.

SPRINT	OBIETTIVO	CONTENUTO
0 Presa in carico	Ricostruire l'ambiente e formarsi un'opinione indipendente	Leggere consegna tecnica e analisi funzionale. <code>dotnet restore && dotnet build</code> . Restore DB demo. Configurare <code>appsettings.Development.local.json</code> con OpenAI key e connection string. Avviare, login con seed, navigare clienti / pratiche / admin-users / studio-chat / audit-log. Verifiche puntuali che ti chiedo espressamente: (a) gli embedding sono davvero <code>double precision[]</code> con similarity in memoria, o pgvector è già in uso? (b) il filtro di assegnazione staff è applicato su tutte le query o no? (c) il pannello comunicazioni e l'export sono gusci come sostengo? Output atteso: una tua valutazione scritta dello stato reale vs. dichiarato, e la tua stima sui punti 09.
1 Fondamenta	Renderlo installabile e non pericoloso	Autorizzazione RAG: filtro per assegnazione applicato uniformemente su <i>tutte</i> le query, più i test che lo dimostrano. pgvector + indice HNSW + retrieval in SQL (blocca Mainardi). Rotazione chiave Plaud dev e password demo del seed. Segreti fuori dalla systemd unit. Permessi filesystem. PostgreSQL su localhost. Verifica disallineamento Documents DB/FS. Blocco reale del budget OpenAI. Astrazione provider AI.
2 Chiudere M4	Sbloccare Studio [C] — il pilota più veloce	Task ereditati #80–#86 e #91–#92 (tabella sotto). Aggiunta ricorrenze sulle scadenze. Work queue cross-pratica.
3 M3 mail	Chiudere Tommaso e Studio [C]	Connessione casella, classificazione, bozza con RAG, conferma obbligatoria, log immutabile d'invio, campagne ricorrenti, regole per organizzazione.
4 M1 hardening	Rendere Mainardi dimostrabile	ClientResolver + coda da attribuire. AzioniRichieste → CaseActivity. Astrazione sorgente trascrizioni. Idempotenza webhook. CallNote nel RAG globale. Onboarding tenant in ore.

Task già scomposti dal precedente sviluppatore

Da riprendere così come sono. Se ne cambi la struttura, dimmi perché.

#	AREA	DESCRIZIONE
80	Workflow	<code>IWorkflowTemplateService</code> + <code>WorkflowTemplateService</code> — implementazione service layer PARZIALE
81	Workflow	<code>ICaseActivityService</code> + <code>CaseActivityService</code> + hook in <code>ICaseService.CreateAsync</code>
82	Pratiche	Redesign <code>CaseEdit.razor</code> — form nuova pratica con selezione template workflow
83	Pratiche	<code>CaseDetail.razor</code> con sezione attività derivate dal workflow
84	Admin	<code>/admin/workflow</code> (lista) e <code>/admin/workflow/{id}</code> (editor template)
85	Nav	Rimuovere voce «Attività» isolata, integrare nel contesto pratica
86	Infra	Build validation + deploy staging dopo completamento workflow
91	Attività	<code>/activities</code> — work queue cross-pratica per lo staff
92	Nav	Aggiornare voce menu Attività

Debito tecnico da sanare

Otto criticità rilevate dal precedente sviluppatore sul suo stesso lavoro. Le riporto integralmente perché sono onestamente dichiarate e perché finché non sono chiuse **non metto dati reali di clienti finali nel sistema**.

#	CRITICITÀ	COSA COMPORTA / COSA FARE
1	Segreti in configurazione server ALTA	La connection string è dentro la unit systemd. Spostare in EnvironmentFile protetto o secret manager.
2	Permessi deploy permissivi ALTA	Molti file sotto <code>/var/www/nexuscore</code> sono world-writable. Proprietario <code>www-data</code> e permessi restrittivi.
3	PostgreSQL esposto ALTA	<code>listen_addresses=*</code> e regola remota ampia in <code>pg_hba.conf</code> . Restringere a localhost.
4	Disallineamento documenti DB/FS MEDIA	Il DB contiene record <code>Documents</code> ma non risultano file fisici nello storage. Verificare prima di qualsiasi migrazione.
5	Documentazione interna obsoleta BASSA	Alcuni file storici e <code>workers/</code> descrivono ingestion/RAG come non implementati. Fonti valide: il codice e <code>analisi-funzionale-software.md</code> .
6	Seed in Staging e Production MEDIA	<code>Program.cs</code> esegue il seed ovunque. Comodo per demo, da rivalutare prima dell'uso reale.
7	Zero test automatici ALTA	Nessun xUnit o equivalente. Con permessi granulari non testati e dati fiscali, è la criticità che mi preoccupa di più commercialmente.
8	AI provider non astratto MEDIA	Client OpenAI istanziati direttamente. Blocca il cambio modello e la risposta alla domanda sulla residenza dei dati.

Quattro punti che aggiungo io, non presenti in quella lista

#	CRITICITÀ	COSA COMPORTA / COSA FARE
9	Autorizzazione RAG non uniforme BLOCCANTE	Nessun test, nessun <code>RagAuthorizationService</code> completo, segregazione staff non applicata su tutte le query. È il punto in cui un nostro difetto diventa un problema legale del cliente. Sprint 1, prima di tutto il resto.
10	Segreti dentro il pacchetto di consegna ALTA	Chiave webhook Plaud dev in chiaro in <code>PIPELINE_RAG.md</code> , password demo costante nel seed. Ruotare entrambe. Trattare il pacchetto come materiale sensibile.
11	Tabelle create a mano in Program.cs MEDIA	Rilevato dall'analisi funzionale. Fuori dalle migration EF significa ambienti che divergono silenziosamente. Da riportare sotto migration.
12	Permessi che proteggono il nulla MEDIA	<code>DataExporter</code> esiste come ruolo ma la funzione export non esiste. Sembra sicurezza, non lo è. Da rimuovere o da implementare — non da lasciare com'è.

NOTA OPERATIVA DAL PACCHETTO DI CONSEGNA

Il server ospita anche **un secondo servizio applicativo indipendente**, non correlato a OVIA e non incluso nella consegna. Non intervenire su quel servizio né sulle sue configurazioni.

Domande aperte e prossimo passo

Cosa mi serve da te, in questo ordine

1. **Sprint 0 fatto e una valutazione scritta.** Non una stima a occhio: voglio sapere se lo stato dichiarato coincide con quello reale, e dove no.
2. **Una posizione sulla decisione «si continua NexusCore».** Se sei d'accordo, dimmelo e chiudiamo. Se non lo sei, voglio le ragioni tecniche prima dell'accordo.
3. **Stima su Sprint 1 e Sprint 2**, che sono i due che sbloccano cassa.
4. **La tua opinione sulle sei domande qui sotto.** Su alcune non ho una preferenza e mi fido della tua.

LE SEI DOMANDE

1. **Casella mail (M3):** IMAP generico, Microsoft Graph o Gmail API? Chi sono i clienti conta: gli studi italiani sono in larga parte su Microsoft 365 o su PEC Aruba/Legalmail. La PEC è un mondo a sé. Quale copriamo in fase 1?
2. **Multi-organizzazione (Tommaso):** i 12 ordini stanno dentro un tenant come 12 «branch», o serve un livello Organization nuovo? Guarda [BranchService](#) e dimmi se regge.
3. **OCR (M2):** ci mettiamo mano in fase 1 o accettiamo che i PDF scansionati restino [Skipped](#) con avviso in UI? Ha impatto diretto sulla percezione «non ricorda niente».
4. **Provider AI:** restiamo su OpenAI o l'astrazione deve prevedere da subito un'alternativa? La domanda sulla residenza dei dati arriverà.
5. **Mobile:** Blazor Server responsive basta per Studio [C], o serve altro? Voglio sapere il costo delle due strade prima di prometterlo.
6. **Test:** da dove parti? Io direi authorization e tenant isolation, perché è dove un bug diventa un problema legale e non un ticket.

Cosa ti serve da me

- Codice sorgente completo, [pacchetto-consegna-nexuscore.zip](#) **ALLEGATO**
- Segreti su canale separato: password DB [nexuscore](#), OpenAI API key, chiave webhook Plaud, credenziali SSH/deploy, credenziali demo da ruotare
- IP server, hostname, informazioni DNS/dominio
- Accesso al mio Plaud per i test di ingestion reale
- Decisioni di prodotto: le prendo io, ma le prendo velocemente. Se aspetti una risposta da me più di 48h, sollecitami

IL PROSSIMO PASSO

Leggi il pacchetto, fai lo Sprint 0, e torniamo a sentirci con la tua valutazione in mano. Non voglio un preventivo prima che tu abbia visto il codice: sarebbe un numero inventato, e li riconosco.

Documento riservato. Contiene informazioni commerciali su clienti reali e prospect di L3 Innovation S.r.l. Non diffondere. Presentazione commerciale del brand in documento separato: [OVIA_02_Presentazione-Brand.pdf](#).